Suppose you have an account with an identifier that multiple users want to update. Alice and Bob want to withdraw money from the account. If you try to synchronize each operation to make sure the account has enough funds for each requested operation, that means only one user can make changes at a time, and the others have to wait until the blocking user is done. This is not very practical and does not correspond to how banks work. This problem was solved a few centuries ago with general ledgers so that changes could be made to an account in different offices by making a new entry in the respective local ledger. At the end of the day, week, or other period, all general ledgers are combined to total the deposits and withdrawals and calculate the total balance. Of course, this approach or pattern isn't limited to accounting. Database logs are another well-known example and can be used as a recovery mechanism. For example, SQL Server transaction logs store all changes to the database, so they can be restored if you find that a user has deleted important data. It is possible to revert to a previous state. However, this is an administrator mechanism, and it is not used by normal users.

In the example of Alice and Bob, there are two ways to store the state. You can replace the value of a database row while blocking, or you can append the change to the transaction logs without blocking, like an old general ledger. These are the options: replace in place versus append-only. Thinking in terms of "replace in place" is generally a poor choice. As Pat Helland put it brilliantly, "The database is a cache of a subset of the log" (*Queue* 13 (9), 2016, https://queue.acm.org/detail.cfm?id=2884038). When you query a store, you typically don't want to read a bunch of logs and aggregate them. That takes time. A deformalized table in a database is quicker to read, although this table—this view—is the product of its history. To